Ingeniería de Software I

Extreme Programming (XP)

Mgtr. Ing. Federico Botti

Agenda - Extreme Programming

ITBA
Instituto Tecnológico de Buenos Aires

1. Introducción
2. Agile Manifesto
3. Extreme Programming
    a. Qué es?
    b. Prácticas
    c. Planificación y Feedback Loop.
    d. XP Lifecycle
4. State of Agile Report
5. Links de interés

Problemas en el desarrollo de software

- Entregas tardías y/o incompletas
- Cancelación de proyectos
- Sistema desactualizado
- Alta tasa de errores
- Reglas de negocio sin entender
- Cambios de requerimientos
- Deserción de programadores
- Ambientes de alto grado de incertidumbre

Fuente: http://www.extremeprogramming.org

Agile Manifesto

Agile Manifesto

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

Fuente: http://agilemanifesto.org/

ITBA
Instituto Tecnológico de Buenos Aires

Extreme Programming

Extreme Programming

Metodología liviana, eficiente de bajo riesgo, flexible, predecible y científica.

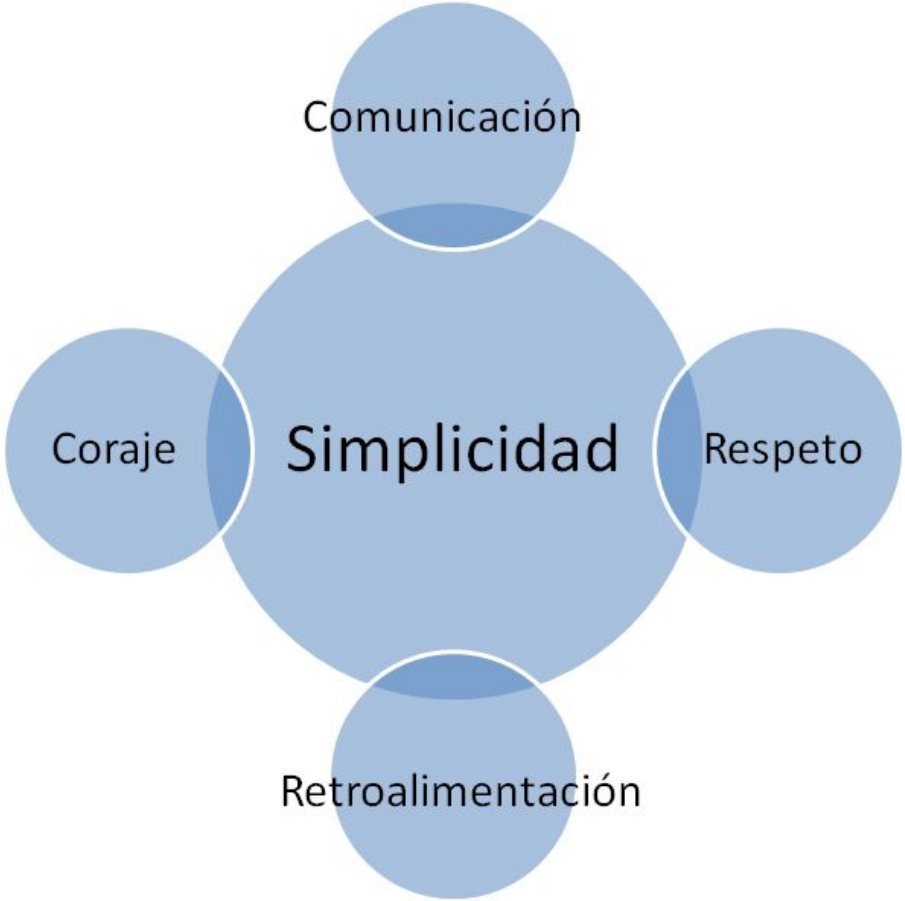
Orientada a **equipos pequeños o medianos**.

Para desarrollar software teniendo como premisa que **los requisitos cambian rápidamente**.

Especialmente diseñadas para ambientes de **gran incertidumbre**.

Fuente: http://www.extremeprogramming.org

Extreme Programming - Valores



Fuente: http://www.extremeprogramming.org

Extreme Programming - Prácticas

- Juego de planificación
- Releases pequeñas
- Metáforas
- Diseño simple
- Testing
- Refactoring
- Pair Programming
- Compartir autoría
- Integración continua
- 40 horas semanales
- Cliente on-site
- Estándares de codificación

Fuente: http://www.extremeprogramming.org/rules.html

Extreme Programming - Prácticas

Juego de planificación

- Determinar rápidamente el alcance de la próxima release combinando las prioridades del negocio y las limitaciones técnicas
- Planificar como máximo dos releases por adelantado
- Como la realidad supera a la planificación, actualizar el plan

Releases pequeñas

- Poner en producción el sistema más pequeño que funcione
- Luego agregar funcionalidad iterativamente
- Cada release debe tener una unidad de coherencia

Metáforas

- Guiar a todo el grupo de desarrollo con una historia simple de cómo debe funcionar el sistema
- Ayuda a comprender los elementos básicos del sistema y la relación entre los mismos
- Debe ser refinada a medida que avanza el proyecto
- Es una vista desde “10000 metros” del sistema

Fuente: http://www.extremeprogramming.org/rules.html

Extreme Programming - Prácticas

Diseño simple

- El sistema debe ser lo más simple posible. Remover toda complejidad.
- El diseño correcto
  - 1. Debe pasar todos los testeos
  - 2. No tiene lógica duplicada
  - 3. Muestra la intención del programador
  - 4. Tiene la menor cantidad posible de métodos y clases
- El futuro es incierto, entonces no diseñar de más para una futura funcionalidad

Testing

- Los testeos unitarios deben ser parte del código
- Los testeos funcionales deben ser parte del programa
- Esto no implica escribir un testeo por cada método. Sólo deben escribirse en el caso que lo amerite

Fuente: http://www.extremeprogramming.org/rules.html

Extreme Programming - Prácticas

Refactoring

- Implica llevar el programa a la forma más simple una vez que se introducen cambios
- Favorece el agregado de nuevos cambios
- Si existe código duplicado debe refactorizarse

Pair Programming

- Una de las personas escribe el código y piensa en la mejor forma de hacerlo
- La otra persona piensa en forma estratégica
  - ¿Es correcta la solución?
  - ¿Hace falta definir otro testeo?
  - ¿Hay forma de simplificar la solución?
- Las parejas deben ir rotando para fomentar el conocimiento del sistema

Fuente: http://www.extremeprogramming.org/rules.html

Extreme Programming - Prácticas

Compartir Autoría

- Cualquiera que ve la oportunidad de agregarle valor a un código debe hacerlo
- No existe el concepto de “dueño del código”
- Mejora los tiempos de corrección de bugs
- La implementación de esta práctica se ve favorecida por la programación de a pares

Integración Continua

- El código debe ser integrado y testeado a intervalos regulares
- Económicamente conviene tener una máquina dedicada a esta tarea
- Cualquier persona que agregue cambios a la release debe cerciorarse que el 100% de los testeos sean exitosos

40hs semanales

- Trabajar más de esta cantidad de horas degrada la capacidad creativa del equipo
- Esto repercute directamente en la cantidad de bugs presentes
- Se permite que una semana se trabaje más horas. Si esta situación se presenta en dos semanas consecutivas entonces el problema no puede solucionarse trabajando más horas

Fuente: http://www.extremeprogramming.org/rules.html

Extreme Programming - Prácticas

Cliente on-site

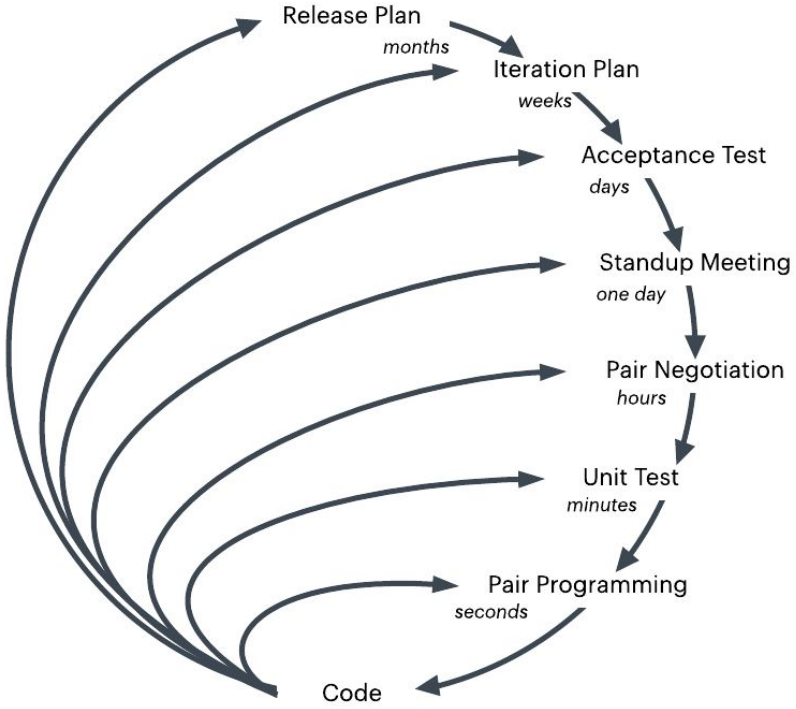
- El cliente debe estar con el equipo, disponible para responder cualquier duda que pueda surgir
- Por cliente se entiende alguien que va a utilizar el sistema cuando el mismo se encuentre en producción
- Mientras no es consultado, el cliente puede seguir realizando su trabajo

Estándares de codificación

- Se deben definir para facilitar la práctica de la autoría compartida de código
- Deben tener como premisa hacer el mínimo trabajo posible
- Debe enfatizar la comunicación
- El equipo debe adoptarlo de común acuerdo

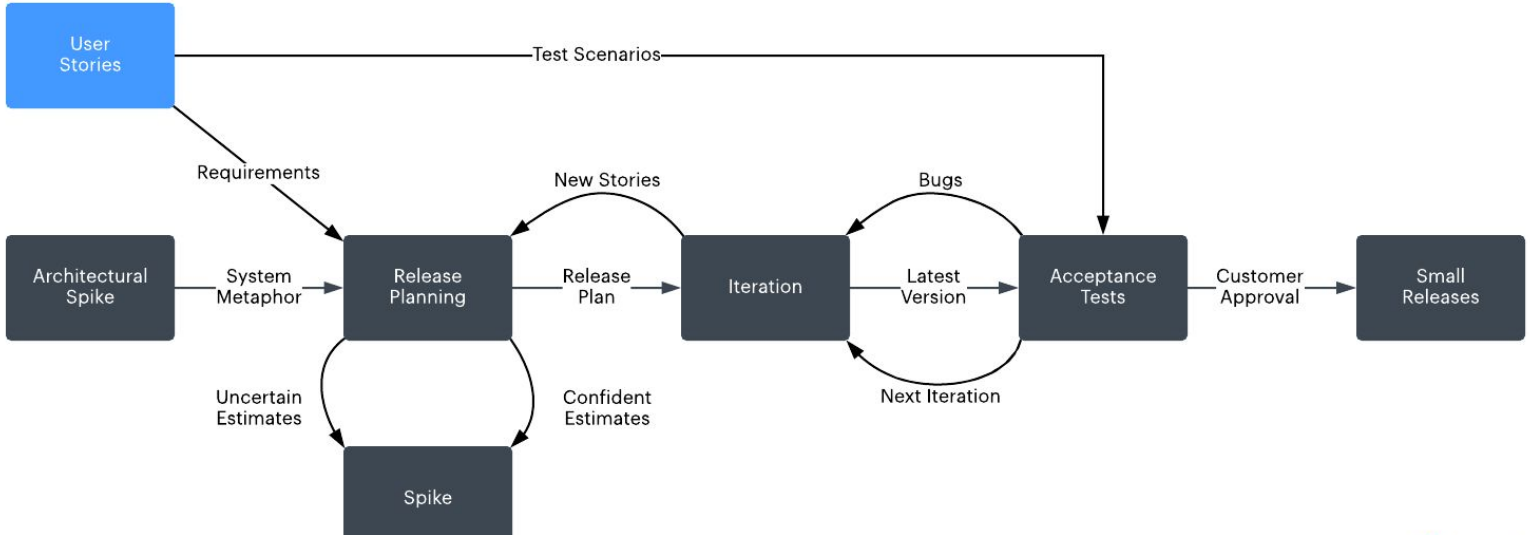
Fuente: http://www.extremeprogramming.org/rules.html

Extreme Programming - Planificación y Feedback loop

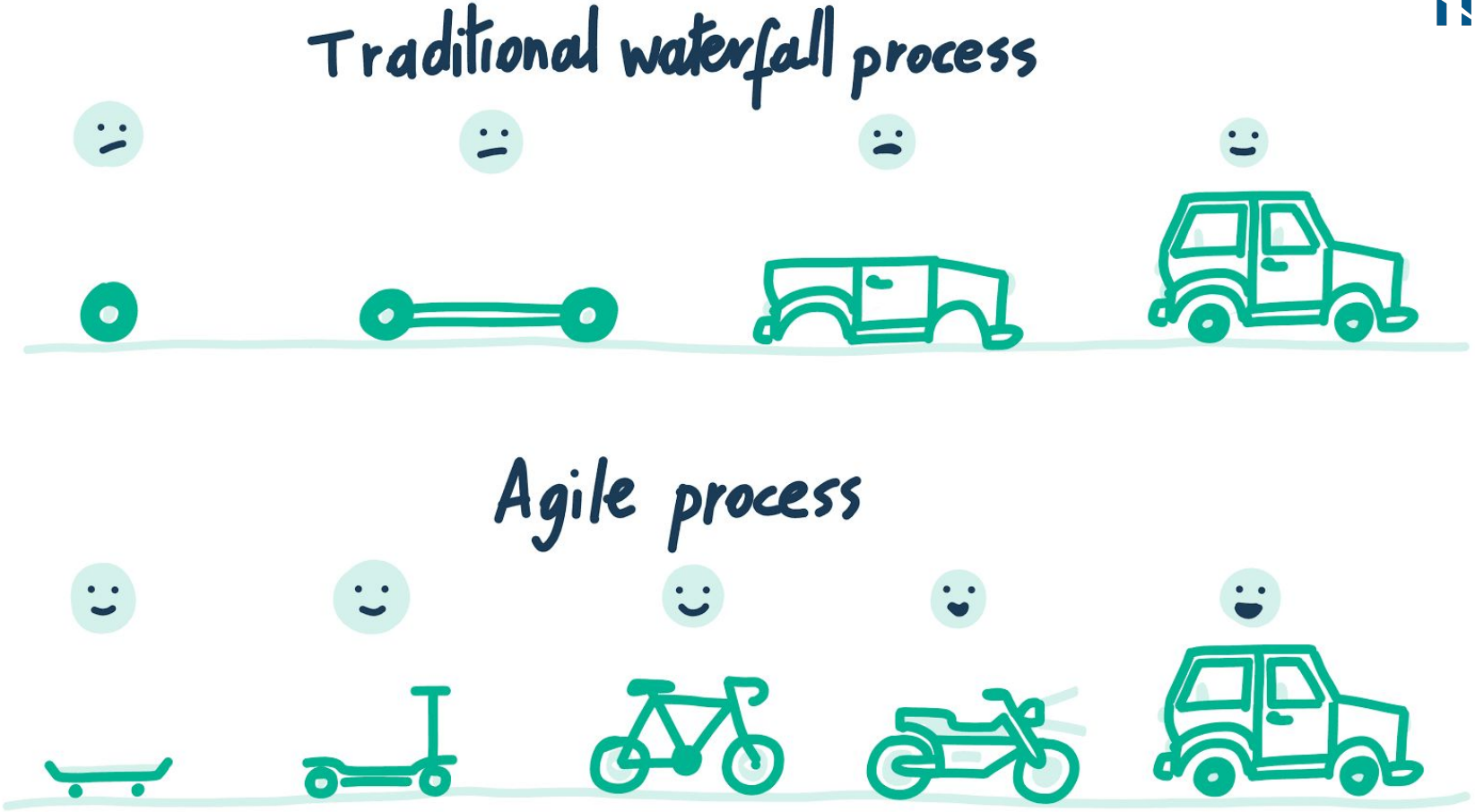


Fuente: http://www.extremeprogramming.org/rules.html

Extreme Programming - Lifecycle



Fuente: http://www.extremeprogramming.org/rules.html


Traditional waterfall process
Agile process

Extreme Programming - Resuelve problemas de software

Entregas tardes

- Iteraciones cortas (1 a 4 semanas)
- Tareas cortas (1 a 3 días)

Cancelación de proyecto

- El cliente elige cual es la menor release que da valor al sistema

Sistemas desactualizados

- Apertura ante los cambios

Alta tasa de errores

- Testeo unitario y funcional
- Especificación iterativa

Reglas de negocio sin entender

- El cliente forma parte del grupo de desarrollo

Cambios de requerimientos

- Iteraciones cortas

Deserción de programadores

- El programador acepta la responsabilidad de estimar y completar su propio trabajo
- Estima y recibe luego el feedback de su estimación
- Contacto entre los integrantes del grupo Los nuevos integrantes del grupo van aceptando más responsabilidades a medida que pasa el tiempo

Fuente: http://www.extremeprogramming.org/rules.html

Agile Benefits

1. Better **product quality**

Testing is integrated during the cycle, with regular checkups, enabling product changes if needed.

2. **Customer Satisfaction**

The Product Owner is involved in the process, making customers part of the project.

3. More **control on projects**

There is more transparency, short meetings and visibility of every step of the project.

4. **Risk reduction**

The risks of total failure are eliminated as there is constant contact. There is freedom to implement new changes through the process, adapting to the client's needs and preferences.

5. **Faster ROI**

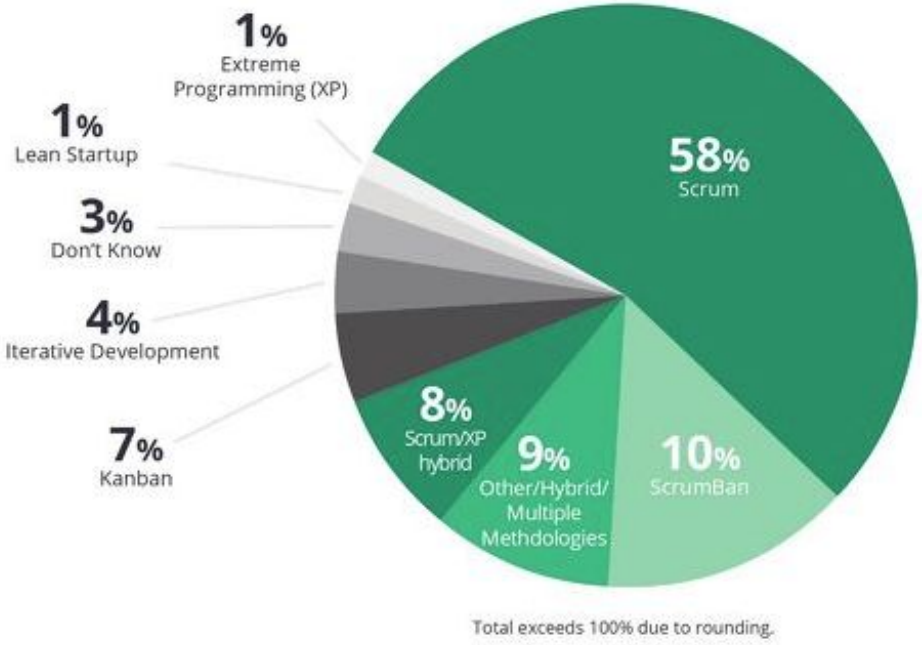
Thanks to agile's iterations, the features are delivered soon, having benefits from the begining while the product is still being developed.

37% faster time to market

16% higher productivity

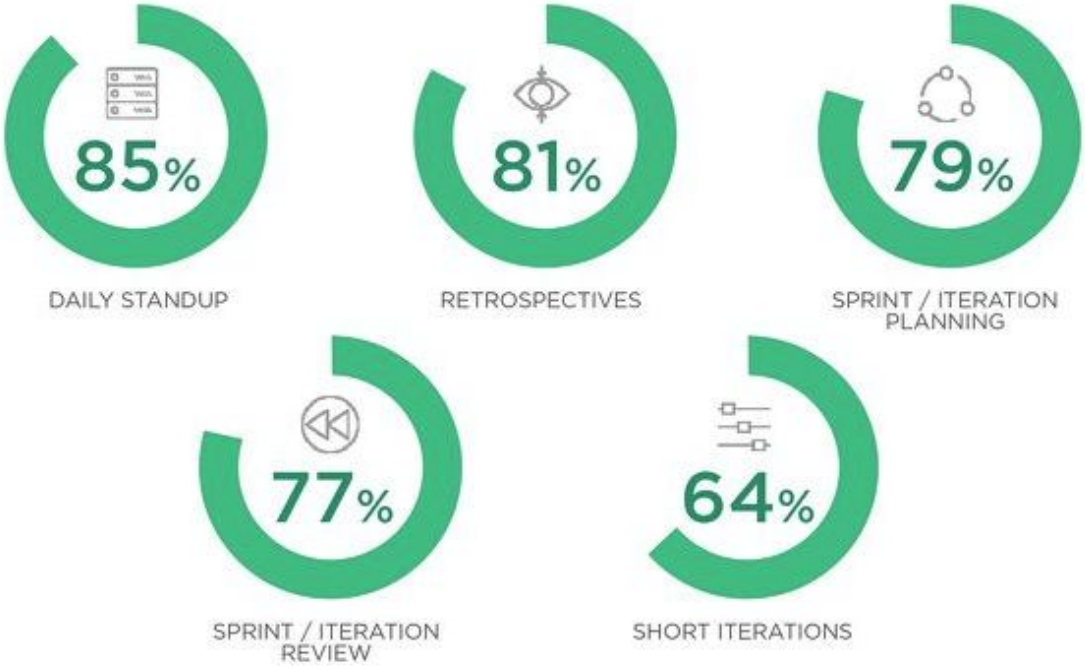
State of Agile Report

Metodologías y su uso



Fuente: https://stateofagile.com/

Metodologías ágiles - Mejores prácticas



*Respondents were able to make multiple selections

Fuente: https://stateofagile.com/

Links de interés

Extreme Programming - Links de interés

- Agile Manifesto
- Agile Alliance
- Extreme programming
- State of Agile
- Caso de Spotify: Parte 1 y Parte 2